

CredVault — Testing Walkthrough

Run the whole system end to end, from nothing to a verified credential.

Written for demoing and recording. It covers what to click and what you should see — not how any of it works internally. For the architecture and threat model, see `Backend-Contracts/README.md`.

The three roles

Everything in CredVault is one of three people. You will play all three, switching MetaMask accounts as you go. Keeping them straight is the whole trick to a clean recording.

ROLE	NEEDS	WHAT THEY DO
INSTITUTION	The registered issuer wallet + Sepolia ETH	Uploads a document, fills in the credential, signs twice, hands over a claim code.
STUDENT	Any second wallet + Sepolia ETH	Accepts or declines the credential, saves the key, shares it with an employer.
VERIFIER	Nothing at all	Checks a credential. No wallet, no account, no permission needed.

READ THIS BEFORE ANYTHING ELSE — WHAT "THE KEY" ACTUALLY IS

The key is **not a login**. The student never needs it to sign in — they sign in by connecting their wallet, full stop.

The key only decides whether the credential's *contents* are readable. Without it, the student still sees the credential exists, who issued it, when, and whether it is valid — and can still accept or decline it. The title and details just show as *"Encrypted credential"*.

The key is generated once, at issuance, and appears in a single dialog on the institution's screen. Copy it then, or the institution has to issue the credential again to produce it. It lives only in the browser you paste it into.

Before you start

- 1 Install MetaMask** and create **two accounts** in it. Account A is the institution, Account B is the student. In MetaMask: avatar menu → *Add account*.
- 2 Switch to Sepolia.** Network dropdown → enable *Show test networks* → pick Sepolia. The app will also offer to switch for you.
- 3 Fund both accounts** with Sepolia ETH from a faucet — e.g. `sepoliafaucet.com` or `cloud.google.com/application/web3/faucet`. A little goes a long way; both accounts need gas to sign.
- 4 Account A must be the registered issuer.** Only a wallet holding `ISSUER_ROLE` can issue. On the live deployment that is `0x8cb784B156c304291cE896AF7881F627BB4633BE` — import that account's private key into MetaMask as Account A. Any other wallet is refused at the door.
- 5 Have a PDF ready** to issue as the certificate. Any PDF works. Under 10 MB.

IF YOU SKIP STEP 4

The institution portal will connect, check the chain, and then refuse with *"This wallet is not authorised as an issuer."* That is the app working correctly, not a bug — but it will stop your recording dead.

Part 1 — Verify something, with nothing

Start here. It needs no wallet and no setup, and it is the clearest demonstration of the point: anyone, anywhere, can check a credential without asking permission from anybody.

1 Open the app and click **Verify** in the top navigation.

2 Choose the **Enter manually** tab.

3 Paste this holder address:

0x8cb784B156c304291cE896AF7881F627BB4633BE

4 And this document hash:

0xd2a65c9bb7163dcba970c037be4a97d8f98ba37969172e127cdecdb123871e1b

5 Click **Verify**.

WHAT YOU SHOULD SEE

A green **Valid** verdict, issued by **CredVault Demo University**, with the issue date and expiry. The credential body reads *"Encrypted — no key supplied"*, which is the system working as designed: validity is public, contents are not.

Expand **Proof** to show the document hash, IPFS CID and contract address. Expand **What is actually stored** to show the raw ciphertext IPFS serves — this is the shot worth lingering on, because it proves the encryption is real rather than claimed.

Part 2 — The institution issues a credential

1 Switch MetaMask to **INSTITUTION** **Account A**.

2 Click **Institutions** in the navigation, then connect. The app checks the issuer role on-chain.

3 On the **Issue** tab, drag your PDF into the dropzone. Its `keccak256` hash appears immediately — computed in your browser; the file is never uploaded.

4 Fill in the **holder wallet address** — this must be Account B's address. Copy it from MetaMask.

5 Fill in holder name, type, title, and optionally an expiry and description.

6 Click **Issue credential**.

You will be asked to sign **twice**, and they are different things:

PROMPT	WHAT IT IS
Signature request	Derives the encryption key. Free, no gas — it is a signature, not a transaction.
Transaction	Writes the credential to the contract. Costs Sepolia gas.

The progress panel walks through the real stages as they happen:

Hashing document Sign to derive the encryption key Encrypting document Uploading to IPFS
Uploading encrypted metadata Confirm the transaction Waiting for confirmation

Part 3 — The hand-off (this is the part that confuses people)

The moment the transaction confirms, a dialog appears: **"Hand this to the student."** It shows a QR code and a block of text that looks like this:

```
{"holder": "0x...", "documentHash": "0x...", "k": "..."}

```

Three fields, and only the third is secret:

FIELD	MEANING
holder	Which wallet the credential belongs to. Public.
documentHash	Which credential it is. Public — it is on-chain already.
k	The decryption key. Not on-chain, not on IPFS, not on any server. This dialog is the only place it ever appears.

DO NOT CLOSE THIS DIALOG BEFORE COPYING

Click **Copy code** (or **Save QR**) and paste it somewhere you can get at it as the student — a notes app is fine. If you close without copying, the only way to get the key back is to issue the credential again.

In the real world the institution would email this, print the QR on the certificate, or hand it over in person. For the demo, copying it to a scratch file is fine — and worth saying out loud while recording, because it explains *why* the key never touches a server.

Part 4 — The student accepts and unlocks

- 1 Switch MetaMask to **STUDENT** **Account B**.
- 2 Click **Students** in the navigation and connect. *No key is needed to get in* — the wallet is the login.
- 3 The new credential appears under **Awaiting your decision**. Note what you can already see without any key: the issuer, the dates, the status. Only the title and details are hidden.
- 4 Click **Accept** and confirm the transaction. This is the consent step — nothing lands in a student's name unless they sign for it. The status moves from *Pending* to *Active*.
- 5 Now unlock it. Click **Claim key**, paste the whole claim code from Part 3 — braces included — and click **Save key**.
- 6 The card changes from *"Encrypted credential"* to the real title and details. The key is stored in this browser only.

WORTH DEMONSTRATING

Try **Decline** on a second issued credential to show the *Rejected* path, and **Back up keys** to show the passphrase-wrapped export — that is the answer to "what happens when the student changes laptop".

Part 5 — The student shares, the employer checks

- 1 On the credential card, click **Share**. You get a QR and a share code — the same shape as before, and it carries the key.
- 2 Copy it, then open **Verify** (a private window is a nice touch — it proves no logged-in state is involved).
- 3 Paste the code into the **Paste code** tab and click **Verify**.
- 4 This time the verdict is **Valid** and the details are decrypted and readable, because the code carried the key.
- 5 Now prove the document. Under **Prove the document**, drop in the same PDF you issued. You should get a match — the file is byte-for-byte the one that was issued.
- 6 Then drop in a **different** PDF. The mismatch alarm fires and shows both hashes side by side. **Record this one**. It is the most convincing thirty seconds in the whole demo.

Part 6 — Revoke, and undo the revoke

- 1 Switch back to **INSTITUTION** **Account A** and open the **Issued** tab.
- 2 Find the credential, click **Revoke**, confirm the dialog, and sign.
- 3 Re-run the verification from Part 5. The verdict is now **Revoked**, and the credential card is overprinted with **VOID**.
- 4 Back on the institution's **Issued** tab, click **Reinstate** and sign.
- 5 Verify once more — it reads **Valid** again.

Worth narrating: every one of those state changes is a public, timestamped, attributable event on a chain anyone can read. A central database can change a record with no external trace. That is the actual argument for building it this way, and it is the strongest thing to end on.

A recording order that flows

- ☐ Verify a credential cold, with no wallet — establishes the premise in 30 seconds
- ☐ Institution issues, showing the live hash and the two signatures
- ☐ The hand-off dialog — pause here and explain the key
- ☐ Student accepts, then pastes the key and watches the card unlock
- ☐ Share to an employer, decrypted this time
- ☐ Drop the wrong PDF, show the mismatch alarm
- ☐ Revoke, re-verify, reinstate

When something goes wrong

WHAT YOU SEE	WHY, AND WHAT TO DO
"This wallet is not authorised as an issuer"	Account A is not the registered issuer wallet. See step 4 of the setup.
Nothing on the student dashboard	You issued to the wrong address, or MetaMask is on a different account. Check the address in the nav matches the one you typed as holder.
"That is not a CredVault claim code"	Partial paste. It must include the opening and closing braces.
"This code carries no key"	You pasted a share code made without a key. Use the claim code from the issuance dialog.
Credential shows but stays encrypted	Expected until the key is saved — and keys are per-browser. Claim it again in this browser, or restore a backup.
Transaction fails or never prompts	Out of Sepolia ETH, or on the wrong network. Top up and confirm the network pill reads Sepolia.
Issuance stalls on "Uploading to IPFS"	The pinning service is unreachable or unconfigured. Verification still works; only issuance needs it.
Verify returns "not found"	Holder address and document hash must be an exact pair. A hash issued to a different holder will not resolve.